Tutorial Sheet — TD 1 bis / Correction Guidelines

→ Solution – Exercise 1

1) **Best case**: **x = A[1]**, found at the first comparison: **1** comparison, **Θ(1)**.

   Worst case: **x** is at position **n** or absent: **n** comparisons, **Θ(n)**.

2) If **x** is at position **i** with probability **1/n**, the algorithm makes **i** comparisons, so the expected number of comparisons is

(1/n) · ∑ (i = 1..n) i = (1/n) · n(n+1)/2 = (n+1)/2 = Θ(n).

The average case has the same order of growth as the worst case here :

➔ If **x** may also be absent with probability **q**, the average becomes **(1−q)(n+1)/2 + q·n**, which is still **Θ(n)**.

3) **Best case: x** is the middle element, found at the first iteration: **Θ(1)**.

   Worst case: **x** is absent (or found only at the last step). Each iteration halves the interval, whose size goes **n**, ≤ **n/2**, ≤ **n/4**, …, **1**, so there are at most **⌊log₂ n⌋ + 1** iterations: **Θ(log n)**. The average case is also **Θ(log n)**, because about half of the elements sit at the deepest level of the search.

Remark: the statement "linear search is **Θ(n)**" is only correct for the worst case. For all inputs together we can only say that its cost is **O(n)** and **Ω(1)**.

→ Solution – Exercise 2

(a) The two outer loops are independent: **n × n** iterations. For each pair **(i, j)**, the while loop doubles **k** from **1** until **k ≥ n**, which takes **⌈log₂ n⌉** iterations, independently of **i** and **j**. The costs multiply: **n × n × Θ(log n) = Θ(n² log n)**

(b) The number of iterations of the while loop depends on **i: j = i** is halved until it reaches **0**, which takes **⌊log₂ i⌋ + 1** iterations. The total is

∑ (i = 1..n) (⌊log₂ i⌋ + 1) = Θ(n log n)

Upper bound: each term is at most **log₂ n + 1**, so the sum is at most **n(log₂ n + 1) = O(n log n)**. Lower bound: the **n/2** terms with **i ≥ n/2** are each at least **log₂ n − 1**, so the sum is at least **(n/2)(log₂ n − 1) = Ω(n log n)**.

Hence **Θ(n log n)** (this is the **log(n!)** entry of the quick reference).

(c) The two inner loops depend on the outer indices, so we sum from the inside out. For fixed i and j the **k-loop** runs **j** times; for fixed **i** the **j-loop** therefore costs **∑ (j = 1..i) j = i(i+1)/2**. Then

∑ (i = 1..n) i(i+1)/2 = n(n+1)(n+2)/6 = Θ(n³)

(d) No. In all three fragments the number of iterations depends only on **n** and never on the values stored in a data structure, so there is a single cost function: the best, worst and average cases coincide. The distinction only matters when a test on the data can stop the loop early or change the path of execution (as in LINEAR-SEARCH of Exercise 1).

Common mistakes to avoid:

- In (a), counting the while loop as n iterations instead of log n, which gives **Θ(n³)**.
- In (b), answering **Θ(n²)** (by treating the inner loop as running **i** times) or **Θ(n)** (by forgetting the sum over **i**); the loop is logarithmic in **i**, not linear.

In (c), answering **Θ(n²)** by reading three nested loops as "two dependent loops plus one"; each additional dependent level adds one power of n.

→ Model Answer (Corrigé Type) [Exercise 3]

A1) Best case: the array is already sorted. For each **i = 2..n** the test **A[j] > key** fails immediately, so there is exactly 1 comparison per iteration and no shift: **n − 1** comparisons in total, **Θ(n)**.

A2) Worst case: the array is strictly decreasing. At iteration **i** the key is smaller than all **i − 1** elements before it, so there are **i − 1** shifts. The total is

∑ (i = 2..n) (i − 1) = n(n−1)/2 shifts, hence Θ(n²).

A3) Each shift removes exactly one inversion (a pair of positions **p < q** with **A[p] > A[q]**), and the sorted array has none, so the number of shifts equals the number of inversions of the input. In a uniformly random permutation, for each of the **n(n−1)/2** pairs of positions, the pair is inverted with probability **1/2** (by symmetry). By linearity of expectation, the expected number of inversions is

(1/2) · n(n−1)/2 = n(n−1)/4 ≈ n²/4.

The number of comparisons is the number of shifts plus at most one extra comparison per iteration (the one that stops the loop), i.e. at most n − 1 more. The average case is therefore **n²/4 + O(n) = Θ(n²)**: half of the worst case, but of the same order.

A4) Not as a statement about all inputs. The best case is **Θ(n)**, the worst and average cases are **Θ(n²)**. The precise statement is: insertion sort runs in **O(n²)** on every input, in **Ω(n)** on every input, and in **Θ(n²)** in the worst case and on average.

Part B) The while loop is logarithmic: **j** takes the values **1, 2, 4, …**, so it runs **L = ⌊log₂ n⌋ + 1** times, independently of **i**. Each of these iterations executes the **k-loop**, which is dependent on **i** and runs **i** times. For a fixed **i** the cost is **L · i**, and

∑ (i = 1..n) L · i = L · n(n+1)/2 = Θ(n² log n)

Part C) The three loops are independent of one another: the for loop runs **n** times, the loop on a runs **⌈log₂ n⌉** times, and for each value of **a**, the loop on **b** runs **⌈log₂ n⌉** times. The costs multiply:

n × Θ(log n) × Θ(log n) = Θ(n log² n)

Common mistakes to avoid:

- In A1, claiming that the best case of insertion sort is **Θ(1)**: every element must at least be compared with its left neighbour, so at least **n − 1** comparisons are always needed.
- In A3, confusing the average case with the worst case, or computing the average of the best and worst cases **((n−1 + n(n−1)/2)/2** is not the expected cost under a random permutation).
- In Part B, replacing the dependent **k-loop** by **n** (the final order is the same here, but the argument is only valid after the sum ∑ **i** has been written down), or counting the **j-loop** as **n** iterations, which gives **Θ(n³)**.
- In Part C, writing **Θ(n log n)**: there are two logarithmic loops, not one, so the factor log n appears twice.